

建立使用者自訂資料型態

- 列舉型態與結構型態

- **15-1: 列舉型態**

何謂列舉(enum)型態.....	15-1
列舉的優點.....	15-3
列舉成員的型別.....	15-4
列舉的預設值.....	15-5
指定列舉型別.....	15-6
使用列舉.....	15-7
透過轉型取得值與字串.....	15-8

- **15-2: 結構型態**

何謂結構(struct)型態.....	15-10
結構的特性.....	15-11
定義結構型態.....	15-12
宣告與設定內容值.....	15-13
比較結構與類別.....	15-16

Module 15

何謂列舉(enum)型態-1

- 列舉是一組具名常數的集合，使用 **enum** 關鍵字定義。
- 列舉成員宣告於 **{ }** 之間，各成員以 **,** 分隔。
- 每個列舉成員在執行時皆為不可變的常數值（唯讀）。
- 可定義於原始程式檔、類別（**class**）或結構（**struct**）中，但不可定義於方法（**method**）內部。
- 可搭配存取修飾詞（如 **public**、**internal**）決定存取範圍。
- 列舉成員的型別皆為整數型別，預設為 **int**。若未指定初值，列舉成員會自動由 **0** 開始遞增。
- 列舉成員的初值不得以變數或方法指定。

何謂列舉(enum)型態-2

```
public enum Season
```

列舉清單
Enumerator List

```
{  
    Spring = 1,  
    Summer = 2,  
    Fall = 3,  
    Winter = 4,  
}
```

列舉成員
Enumerator

列舉的優點

- 提高程式的可讀性。
 - 使用具意義的名稱取代數字常數，程式邏輯更清楚。
- 易於理解與維護。
 - 以成員名稱代表固定常數值，閱讀時不需記憶數值。
 - 若需變更常數值，只需在列舉定義中修改一次即可。
- 避免產生**Magic number**。
 - 不再於程式中出現難以理解的純數值常數。

```
int InSeason= 1;  
if (InSeason == 2)  
    Console.WriteLine("夏天");
```

使用列舉 →

```
int InSeason= 1;  
if (InSeason == (int)Season.Summer)  
    Console.WriteLine("夏天");
```

列舉成員的型別

- 列舉成員的型別均為整數型別，允許使用：
 - `byte`、`sbyte`、`short`、`ushort`、`int`、`uint`、`long`、`ulong`
- 若未宣告指定型別，預設為`int`。

列舉的預設值

- 列舉是常數的集合，可指定常數初值。
- 若未指定初值，則預設常數依序由0開始。

```
public enum Season
{
    Spring = 1,
    Summer = 4,
    Fall = 7,
    Winter = 10,
}
```

指定初值

```
public enum Season
{
    Spring ,    //0
    Summer,    //1
    Fall ,     //2
    Winter ,   //3
}
```

未指定初值

指定列舉型別

- enum 列舉名稱：型態

```
{  
    列舉成員，  
    ..... ,  
}
```

```
public enum MyEnumList : long  
{  
    MyByte1 = 10000000,  
    MyByte2 = 2000000,  
    MyByte3 = 455451,  
}
```

使用列舉

- 試著輸出列舉。

```
enum Season
{
    Spring = 1,
    Summer = 2,
    Fall = 3,
    Winter = 4,
}
```

```
Console.WriteLine(Season.Summer);
Console.WriteLine((int)Season.Summer);
```

透過轉型取得值與字串-1

- 透過將列舉轉型，可以得到列舉值或列舉名稱字串。

```
string sumstring = Season.Summer.ToString(); //Summer  
int sumint = Convert.ToInt32(Season.Summer); // 2
```

透過轉型取得值與字串-2

- 範例：

```
enum Season
{
    Spring = 1,
    Summer = 2,
    Fall = 3,
    Winter = 4,
}
```

```
private void button1_Click(object sender, EventArgs e)
{
    int x = int.Parse(textBox1.Text);
    // if ... else ...
    switch (x)
    {
        case (int)Season.Spring:
            MessageBox.Show("春天");
            break;
        case (int)Season.Summer:
            MessageBox.Show("夏天");
            break;
        case (int)Season.Fall:
            MessageBox.Show("秋天");
            break;
        case (int)Season.Winter:
            MessageBox.Show("冬天");
            break;
    }
}
```

何謂結構(struct)型態

- 是一種自訂資料型態，用來封裝多個彼此相關、但型態不同的成員（欄位、屬性）。
- 可將屬於同一概念的多項資料整合為一個整體，讓資料處理更有組織、可讀性更高。
 - 一個產品明細中包含產品名稱(string)、單價(int)、生產日期(DateTime)，其中含有多種型態，此時可將產品(含名稱、單價、生產日期)製成一個結構，我們就能以一個變數來處理產品明細。

結構的特性

- 結構內的物件名稱前要加上**public**才可被外界存取。
- 無法在定義時指定初值給屬性(**Property**)或資料欄位(**Field**)。
 - 除非欄位宣告為**const**或**static**。
- 建構方法必須設定每個屬性內容值。
 - 建構方法：類別或結構實作時首先執行的方法。
- 結構可不用**new**實作，但結構設有建構方法、屬性時就必須實作**new**(結構類型具現化)。
- 結構不可定義於方法 (**method**) 內部，只能存在於命名空間 (**namespace**) 、類別 (**class**) 或其他結構中。

定義結構型態

```
struct MyStruct
{
    public int Aint;           //欄位
    public string Bsrting;    //欄位
    public int Ap { get; set; } //屬性
    MyStruct(int s)           //建構函式(子方法)
    {
        Aint = 11;
        Ap = 22;
        Bsrting = "哈囉是我啦，我結構啦";
    }
    public void vo()          //方法
    {
        Console.WriteLine(Ap);
    }
}
```

宣告與設定內容值-1

```
struct Product
{
    public string Name;
    public int Price;
}
```

```
private void button1_Click(object sender, EventArgs e)
{
    Product ns;
    ns.Name = "Nintendo Switch";
    ns.Price = 9780;

    Product ps4;
    ps4.Name = "Play Station 4 PRO";
    ps4.Price = 13980;

    label1.Text = "產品名稱：" + ns.Name + "\n價格：" + ns.Price;
    label2.Text = "產品名稱：" + ps4.Name + "\n價格：" + ps4.Price;
}
```

宣告與設定內容值-2

- 當結構擁有方法或屬性時，必須以new建立實作。

```
struct Product
{
    public string Name;
    public int Price;
    public string Developer { get; set; }
    public Product(string name, int price, string developer)
    {
        Name = name;
        Price = price;
        Developer = developer;
    }
}
```

宣告與設定內容值-3

- 當結構擁有方法或屬性時，必須以new建立實作。

```
private void button1_Click(object sender, EventArgs e)
{
    Product ns = new Product("Nintendo Switch", 9780, "Nintendo");

    Product ps4 = new Product("Play Station 4 PRO", 13980, "SONY");

    label1.Text = "產品名稱：" + ns.Name + "\n開發商：" + ns.Developer;
    label2.Text = "產品名稱：" + ps4.Name + "\n開發商：" + ps4.Developer;
}
```

比較結構與類別

	結構(struct)	類別(class)
定義初值	不可定義初值	可定義初值
型別	實值型別 (Value Type)	參考型別 (Reference Type)
宣告使用	可直接使用，不須建立物件 (除非結構中有方法或屬性)	必須new建立物件
繼承性	不可繼承及被繼承	可繼承及被繼承